ITL STREAMING PORTAL · DATEN & SYSTEMMANAGEMENT

JSON und XML



Strukturierte Daten im ITL-Streaming-Portal

Datenformate verstehen, vergleichen und sinnvoll einsetzen

JSON · XML · API

Strukturierte Daten · zuverlässige Integrationen · aktuelle Inhalte

Made with GAMMA

FORMATE · GEMEINSAME GRUNDIDEE

Zwei Formate, ein Zweck

Beide speichern und übertragen strukturierte Daten. Der Unterschied liegt vor allem darin, wie diese Daten beschrieben und eingesetzt werden.



JSON

Leichtgewichtiges, textbasiertes Datenformat

WEB



XML

Erweiterbare textbasierte Auszeichnungssprache

STANDARD



Gemeinsamer Zweck

Vom Browser über das Backend bis zur Datenbank

DATENFLUSS

Made with GAMMA

JSON · SYNTAX

JSON liest Daten als Struktur

1. Objekte stehen in geschweiften Klammern.
2. Schlüssel und Werte bilden Paare.
3. Arrays stehen in eckigen Klammern.
4. Objekte können verschachtelt werden.

FILMBEISPIEL

Ein Objekt mit drei Schlüssel-Wert-Paaren

```
{"titel":"Neonstadt",
"erscheinungsjahr":2024,
"verfuegbar":true}
```

Made with GAMMA

JSON · BAUSTEINE

Die sechs JSON-Datentypen

01

String

"Neonstadt"

02

Number

2024

03

Boolean

true

04

Object

{"genre":"Sci-Fi"}

05

Array

["Drama","Mystery"]

06

Null

null

Datum und Uhrzeit werden meist als String übertragen, z. B. `"2024-09-01T20:15:00Z"`.

Made with GAMMA

JSON · WEB-APIS

JSON ist die Sprache vieler Web-APIs

Browser

Sendet eine Anfrage

Server

Verarbeitet die Anfrage

JSON-Antwort

Liefert Daten oder einen Fehler


```
{"status":"ok",
"film":{"id":42,"titel":"Neonstadt"}}
```



```
{"status":"error",
"code":"FILM_NOT_FOUND",
"message":"Film nicht gefunden"}
```


Made with GAMMA

JSON · QUALITÄT & SICHERHEIT

JSON Schema schützt die Datenqualität

Syntaxprüfung

Ist das JSON formal korrekt?

Fachliche Validierung

JSON Schema prüft Pflichtfelder, Datentypen und Wertebereiche.

Serverseitige Prüfung

SQL-Injection verhindern. Keine Passwörter oder unnötigen Personendaten übertragen.

JSON Schema beschreibt, welche Daten erlaubt und erforderlich sind. Eingaben validieren und Berechtigungen serverseitig prüfen.

Made with GAMMA

XML · SYNTAX

XML beschreibt Daten mit Tags

1. Elemente haben öffnende und schließende Tags.
2. Attribute ergänzen Informationen.
3. Jedes Dokument hat genau ein Wurzelelement.
4. XML-Deklaration und UTF-8 schaffen Klarheit.

Ein XML-Dokument für denselben Film

```
<?xml version="1.0" encoding="UTF-8"?>
<film id="42">
  <titel>Neonstadt</titel>
  <jahr>2024</jahr>
  <verfuegbar>true</verfuegbar>
</film>
```

Made with GAMMA

HTML und XML im direkten Vergleich

Beide verwenden eine Tag-Syntax - ihre Ziele sind aber unterschiedlich.

HTML: Darstellung einer Seite

```
<!doctype html>
<html>
  <body>
    <h1>Neonstadt</h1>
    <p>Ein Film aus dem Katalog.</p>
  </body>
</html>
```

HTML-Tags haben eine festgelegte Bedeutung für die Darstellung im Browser.

XML: Strukturierte Daten

```
<film id="42">
  <titel>Neonstadt</titel>
  <beschreibung>Ein Film
    aus dem Katalog.</beschreibung>
</film>
```

XML-Tags werden selbst gewählt und beschreiben Bedeutung und Struktur.

Merksatz: HTML beschreibt, wie Inhalte dargestellt werden. XML beschreibt, welche Daten vorhanden sind und wie sie strukturiert sind.

XML · PRÜFUNG & VERARBEITUNG

XML: wohlgeformt oder gültig?



Wohlgeformt

Tags sind korrekt verschachtelt und geschlossen.

SYNTAX



Gültig

Das Dokument erfüllt DTD- oder XSD-Regeln.

REGELN

Namespaces

Vermeiden Namenskonflikte.

DOM

Lädt den Baum.

SAX & Streaming

Lesen ereignisbasiert und verarbeiten große Kataloge speichersparend.

Made with GAMMA

Eine vollständige XML-Datei

Namespaces verhindern Verwechslungen, wenn verschiedene XML-Vokabulare zusammenkommen.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<daten xmlns:videothek="https://beispiel.at/filme"
       xmlns:bibliothek="https://beispiel.at/buecher">
  <!-- Beide Bereiche verwenden den Tag titel -->
  <videothek:film id="42">
    <videothek:titel>Neonstadt</videothek:titel>
    <videothek:jahr>2024</videothek:jahr>
  </videothek:film>
  <bibliothek:buch id="7">
    <bibliothek:titel>XML Grundlagen</bibliothek:titel>
    <bibliothek:autor>M. Beispiel</bibliothek:autor>
  </bibliothek:buch>
</daten>
```

Was ist ein Namespace?
Ein eindeutiger Namensbereich für XML-Tags.

Präfix
`videothek` und `bibliothek` zeigen, aus welchem Vokabular ein Tag stammt.

Warum nötig?
Beide Bereiche könnten einen Tag `<titel>` besitzen. Der Namespace verhindert Verwechslungen.

Die URI
Die Adresse ist eine eindeutige Kennzeichnung. Sie muss nicht aufrufbar sein.

Weitere Bausteine
XML-Deklaration, ein Wurzelelement, Tags, Attribute, Kommentare und korrekte Verschachtelung.

JSON und XML im direkten Vergleich

Dieselben Filmdaten: JSON `{"titel":"Neonstadt","jahr":2024}` · XML `<film><titel>Neonstadt</titel><jahr>2024</jahr></film>`

Kriterium	JSON	XML
Lesbarkeit	kompakt	ausführlicher
Dateigröße	meist kleiner	meist größer
Syntax	Klammern, Paare	Tags, Attribute
Validierung	JSON Schema	DTD / XSD
Web-APIs	sehr verbreitet	möglich, aber schwerer
Namespaces / Metadaten	eingeschränkt	stark
Einsatz	Web, Apps, APIs	Kataloge, Standards, Integrationen

Made with GAMMA

Der Film „Neonstadt“ in JSON und XML

Beide Formate speichern dieselben Informationen, verwenden aber unterschiedliche Syntax.

JSON

```
{
  "id": 42,
  "titel": "Neonstadt",
  "jahr": 2024,
  "verfuegbar": true
}
```

- kompakt
- Schlüssel-Wert-Paare
- sehr verbreitet in Web-APIs

XML

```
<film id="42">
  <titel>Neonstadt</titel>
  <jahr>2024</jahr>
  <verfuegbar>true</verfuegbar>
</film>
```

- Tags und Attribute
- ausführlicher
- starke Validierung mit XSD möglich

Fazit: JSON eignet sich oft für kompakte Web-Kommunikation. XML ist besonders nützlich bei geprüften Katalogen, Standards und Integrationen.

ITL STREAMING PORTAL · ARCHITEKTUR

Im ITL-Streaming-Portal ergänzen sich beide

JSON ist meist kompakter und für moderne Web-APIs praktischer. XML ist ausführlicher, aber stark bei Validierung, Namespaces und Standards.



Web

Suchfilter, Bewertungen, Watchlists

JSON



APIs

Moderne Web-Kommunikation

JSON



Kataloge

Importe und Exporte

XML



Anbieter

Vorgaben mit XSD

XML

Passendes Format statt Konkurrenz: JSON für kompakte Web-APIs, XML für geprüfte Kataloge, Standards und Integrationen.

Made with GAMMA